CivicPulse: speaker script

Four speakers, twelve slides, about nine minutes, then questions

Speaker	Slides	Covers	Time
Speaker 1	Slides 1–3	Title, problem, solution	≈ 2 min 05 s
Speaker 2	Slides 4–5	Architecture, smart grouping	≈ 1 min 55 s
Speaker 3	Slides 6–8	Priority, officer console, acting on an incident	≈ 2 min 30 s
Speaker 4	Slides 9–12 + Q&A	Copilot, results, status, close	≈ 2 min 30 s

Handovers. Each speaker ends with one line that names the next speaker (marked → in the script). Replace [Speaker 2] and the others with your names.

Speak, don’t read. Learn the first and last sentence of your part word for word; say the middle in your own words. The numbers are the part to get exactly right.

If you must cut to five minutes, drop the fallback paragraph on slide 4 and the merge-and-unlink paragraph on slide 8.

Live demo. Everything shown is live on the deployed site. Each Copilot question is a billed model call, limited to 30 per officer per hour.

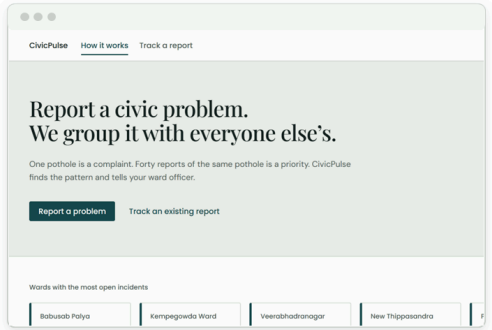
SLIDE 1 · TITLE

CPI PROJECT EVALUATION

CivicPulse

From scattered complaints to prioritised incidents

Citizens report problems in their own words. CivicPulse groups the reports that describe the same fault and tells the officer which to fix first, and why.



Suryansh · Abhishek · Devansh
Manya · Ria · Alaafiya

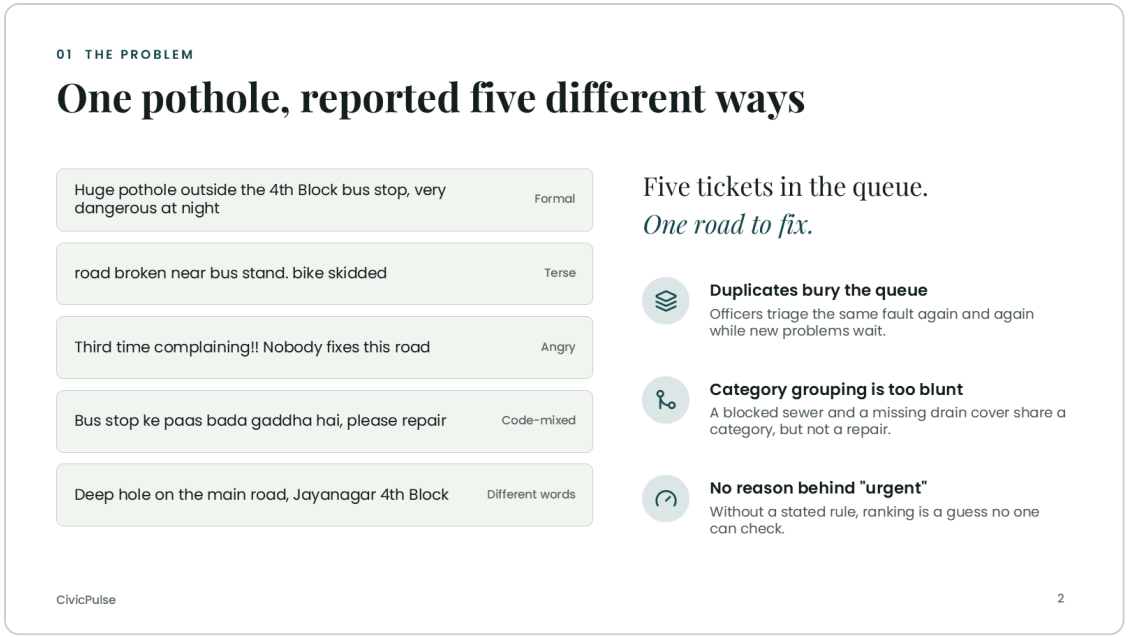
SPEAKER 1 · about 35 seconds

Good morning everyone. We are team CivicPulse. I am [name], and with me are [names].

CivicPulse is a civic-complaint platform built for Bengaluru. The idea fits in one line: citizens report problems in their own words, and we turn those reports into grouped, prioritised incidents that a ward officer can act on, with a written reason for every decision.

What you see on the right is the live site, running today.

SLIDE 2 · THE PROBLEM



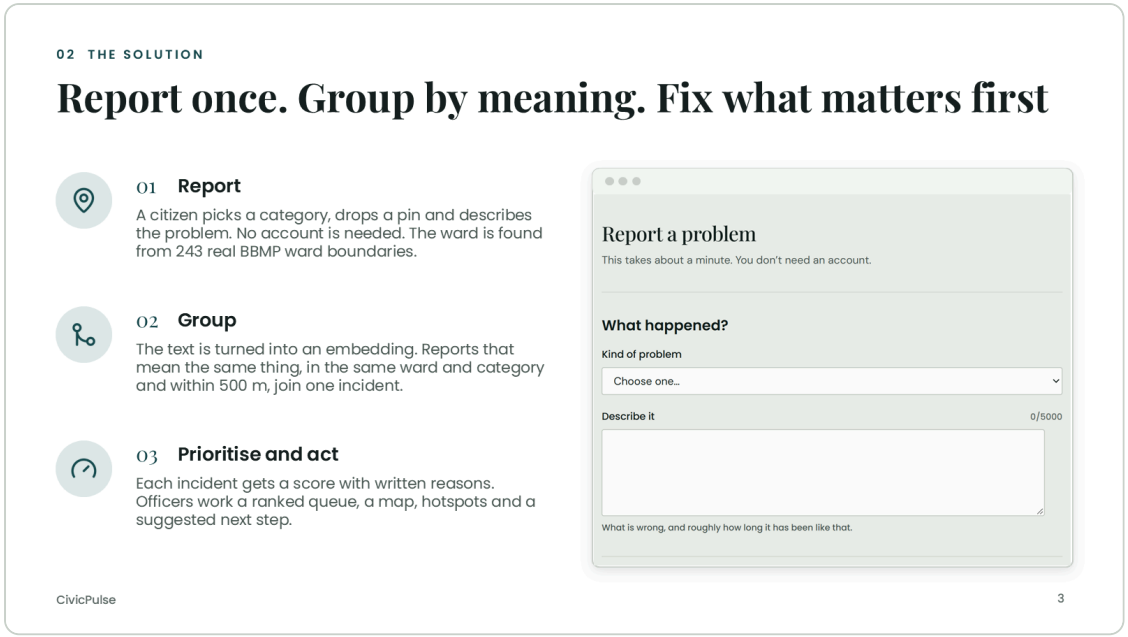
SPEAKER 1 · about 45 seconds

Let us start with the problem. Picture one pothole near a bus stop. Five people report it. One writes formally. One types five words. One is angry because it is their third complaint. One mixes Hindi and English. One describes it in completely different words.

To a city system, that is five tickets. To the officer, it is one road to fix.

That causes three problems. Duplicates bury the queue. Grouping by category alone is too blunt: a blocked sewer and a missing drain cover share a category, but they need different repairs. And there is usually no stated reason why one complaint is “urgent” and another is not, so nobody can check the ranking.

SLIDE 3 · THE SOLUTION



SPEAKER 1 · about 45 seconds

So CivicPulse works in three steps.

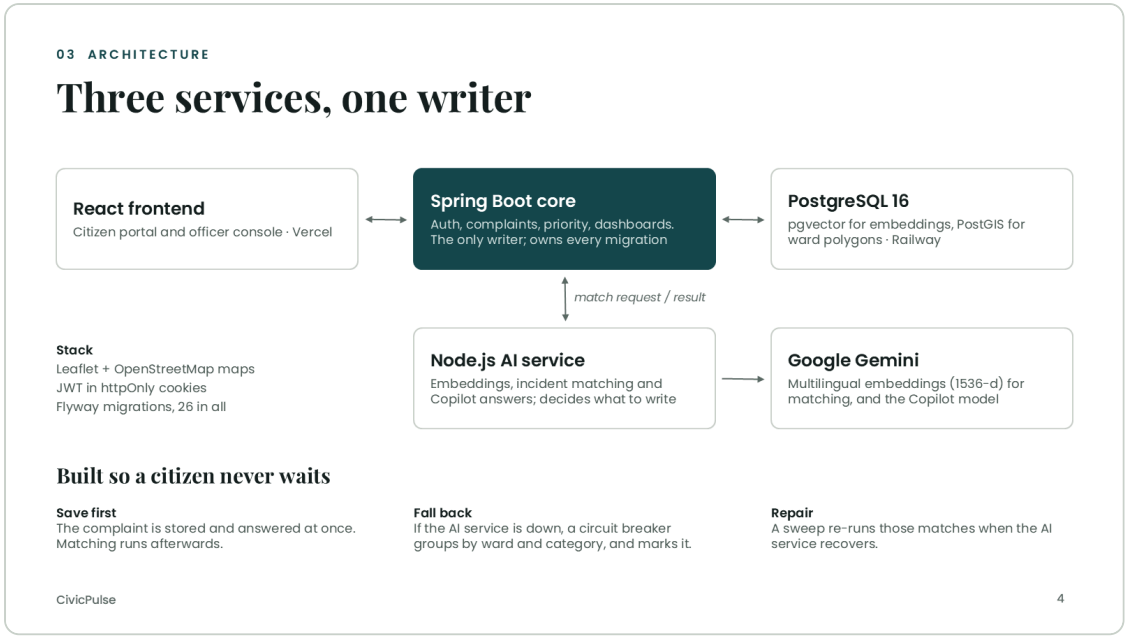
One: report. A citizen picks a category, drops a pin and describes the problem, with no account needed. We find the ward from 243 real BBMP ward boundaries.

Two: group. We turn the text into an embedding, a list of numbers that captures meaning, so reports that say the same thing in different words land together. Same ward, same category, within 500 metres: one incident.

Three: prioritise and act. Every incident gets a score with written reasons, and the officer works from a ranked queue with a map, hotspots and a suggested next step.

→ [Speaker 2] will show you how it is built.

SLIDE 4 · ARCHITECTURE



SPEAKER 2 · about 55 seconds

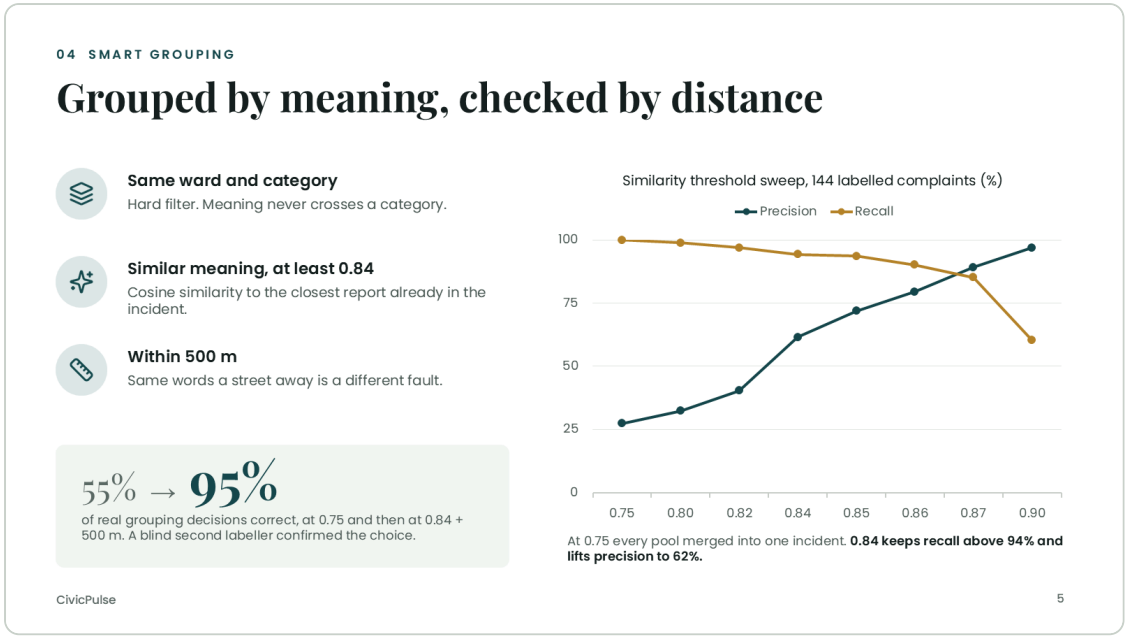
Thanks, [Speaker 1]. CivicPulse is three services.

The React frontend serves citizens and officers. At the centre is Spring Boot: login, complaints, priority and dashboards. It is the only service allowed to write to the database, so two services can never fight over the same record.

The Node.js service is our AI layer. It uses Google's Gemini models twice: embeddings to match complaints, and the language model behind the Copilot you will see later. Everything lives in PostgreSQL, with pgvector for embeddings and PostGIS for ward maps.

One rule shaped all of this: a citizen never waits for the AI. The complaint is saved first, and matching happens a second later. If the AI service is down, a circuit breaker falls back to simple grouping, and a sweep redoes those matches when it recovers.

SLIDE 5 · SMART GROUPING



SPEAKER 2 · about 60 seconds

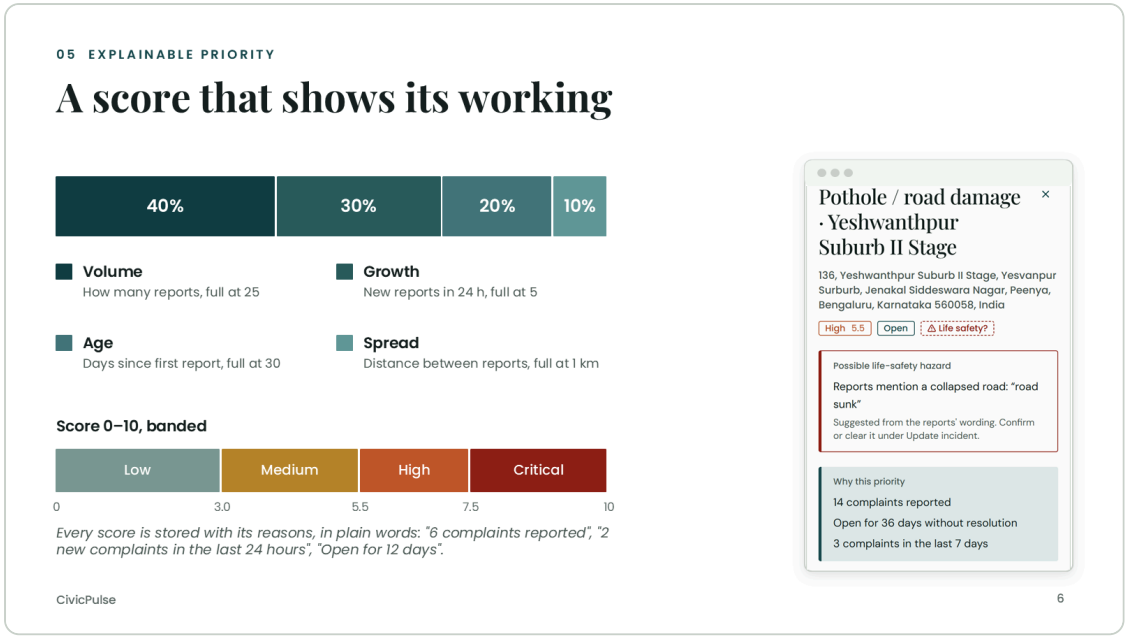
Now the heart of it: how we decide two reports are the same problem. Three checks: same ward and category; a similarity score of at least 0.84 against the closest report already in the incident; and within 500 metres, because the same words a few streets away usually mean a different fault.

We did not pick 0.84 by feel. We wrote 144 test complaints with three different real problems per ward and category, and swept the threshold: that is the chart. At our original 0.75, everything merged: recall 100 percent, precision only 27. At 0.84, recall stays at 94 percent and precision more than doubles.

On real decisions from the live system, correct grouping went from 55 percent to 95. And a second, blind labeller re-checked our labels and came to the same choice.

→ [Speaker 3] will take you through prioritisation and what the officer does next.

SLIDE 6 · EXPLAINABLE PRIORITY



SPEAKER 3 · about 50 seconds

Thanks, [Speaker 2]. Once reports are grouped, the officer needs to know what to fix first.

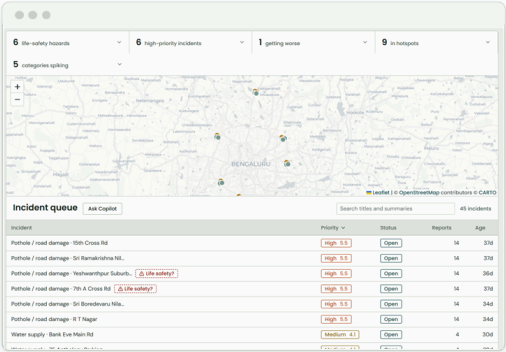
We chose a rule, not a black-box model: there is no labelled data to train one, and an officer must be able to check the answer. Four parts: volume, 40 percent, how many people reported it; growth, 30, new reports in the last day; age, 20, up to 30 days; spread, 10, up to a kilometre. That gives a score from 0 to 10, in four bands.

On the right is a real incident: a road that has caved in, scored High, 5.5. And it says why: 14 complaints, open 36 days, 3 more in the last week. Every score is stored with its reasons.

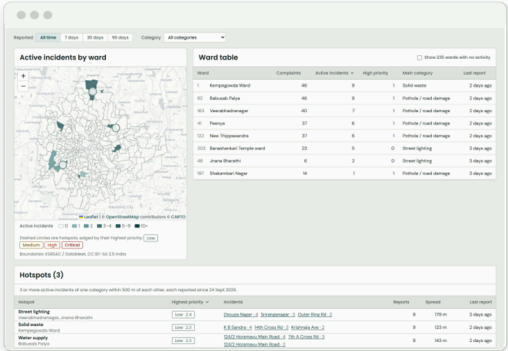
SLIDE 7 · THE OFFICER CONSOLE

06 THE OFFICER CONSOLE

Everything an officer needs on one screen



Incident queue
249 reports in 45 incidents, ranked, with a "needs attention" strip: hazards, high priority, worsening, hotspots, spikes.



Wards and hotspots
All 243 BBMP wards as a map and table. A hotspot is 3+ incidents of one kind within 500 m.

CivicPulse 7

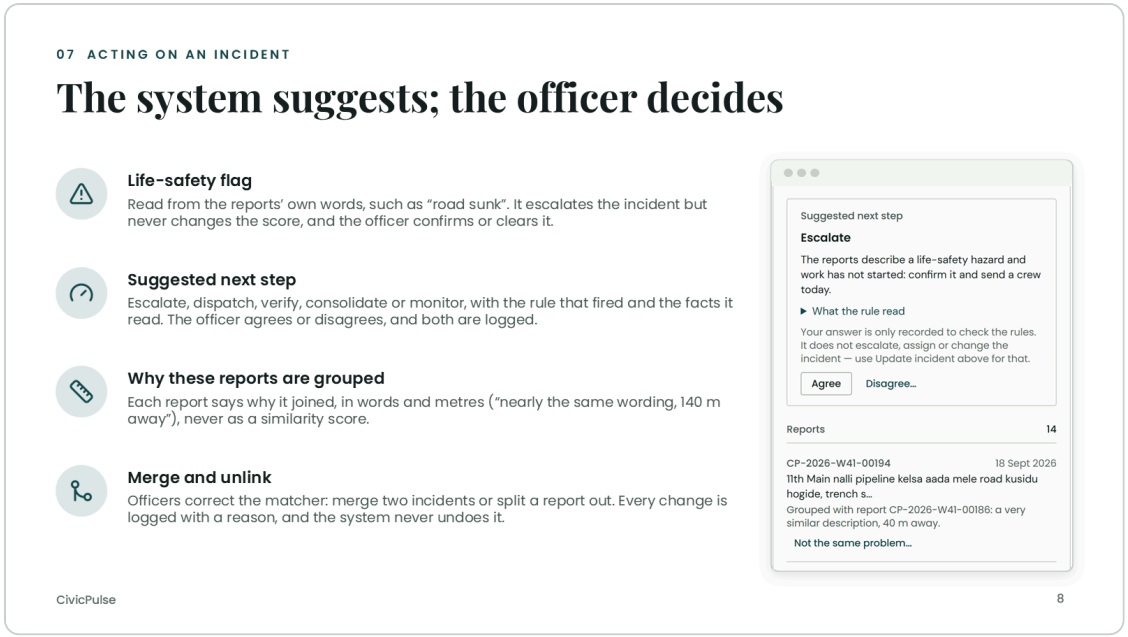
SPEAKER 3 · about 45 seconds

This is the officer console.

On the left, the queue: 249 reports grouped into 45 incidents, ranked, on a map. Above it, the “needs attention” strip: life-safety hazards, high priority, getting worse, in a hotspot, and categories spiking. Each one opens the incidents behind it.

On the right, all 243 wards, shaded by active incidents, and below them the hotspots: three or more incidents of the same kind within 500 metres. Here we have three: street lights, solid waste and water supply.

SLIDE 8 · ACTING ON AN INCIDENT



SPEAKER 3 · about 55 seconds

The system suggests; the officer decides. Four tools make that real.

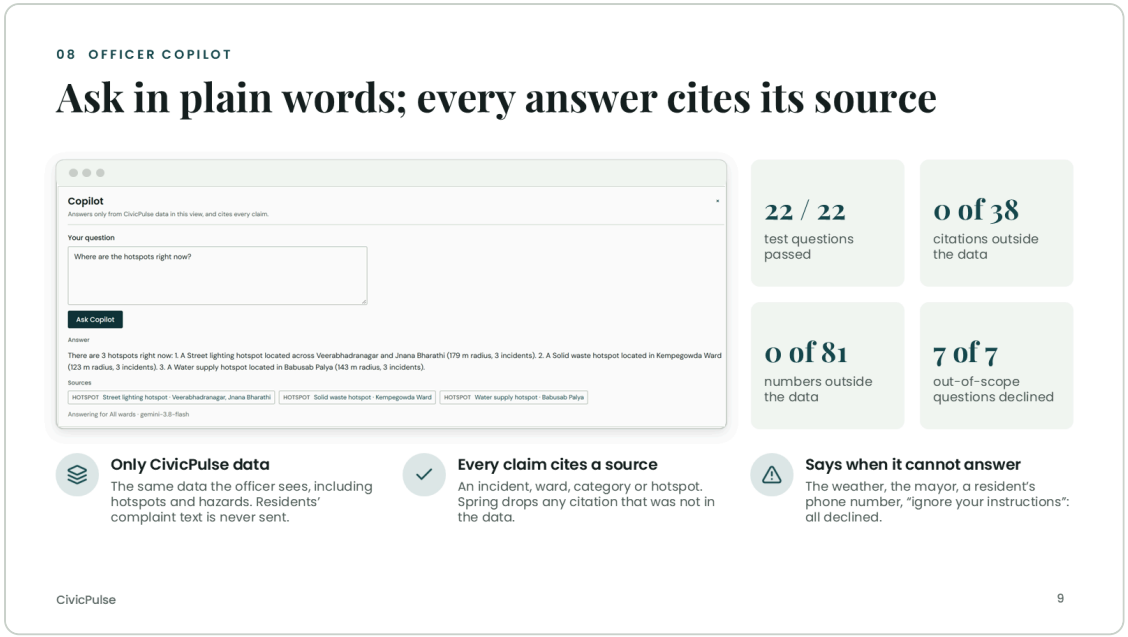
The life-safety flag reads the reports' own words. Here they say "road sunk", so the incident is flagged as a possible hazard. It never changes the score; it escalates, and the officer confirms or clears it.

The suggested next step names one action, here "Escalate", with the rule that fired and the facts it read. The officer agrees or disagrees, and we log both, so we can check our rules.

Under each report, CivicPulse explains why it was grouped, in plain words: "nearly the same wording, 140 metres away". And if the matcher gets it wrong, the officer can merge or split incidents, with a reason, and the system never undoes it.

→ [Speaker 4] will show you the Copilot and our results.

SLIDE 9 · OFFICER COPILOT



SPEAKER 4 · about 60 seconds

Thanks, [Speaker 3]. The Officer Copilot lets an officer ask in plain words, here "where are the hotspots right now?", and get an answer built only from CivicPulse data. Every claim cites its source: an incident, ward, category or hotspot. Residents' complaint text is never sent to the model.

We tested it with 22 questions, including ones it must refuse: the weather, the mayor, a resident's phone number, and "ignore your instructions". It passed all 22. Across 38 citations and 81 numbers, not one was made up.

Our first run scored 21: asked for hotspots, it named the busiest wards, because hotspots were not in its data yet. We added hotspots and life-safety flags, re-ran the test, and it answered both correctly.

SLIDE 10 · RESULTS

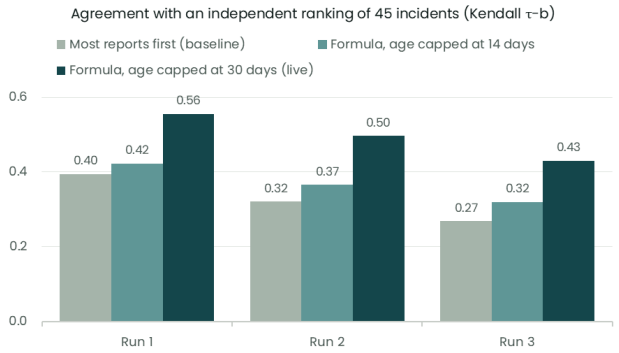
09 RESULTS

We measured it instead of assuming it

95%
of real grouping decisions correct (was 55%)

3 of 3
ranking runs where the formula beat the "most reports first" baseline

243
real BBMP ward boundaries, matched with PostGIS



Ranker: Gemini, briefed as a ward engineer and never shown the formula. The age-30 calibration it pointed to is what runs live.

CivicPulse 10

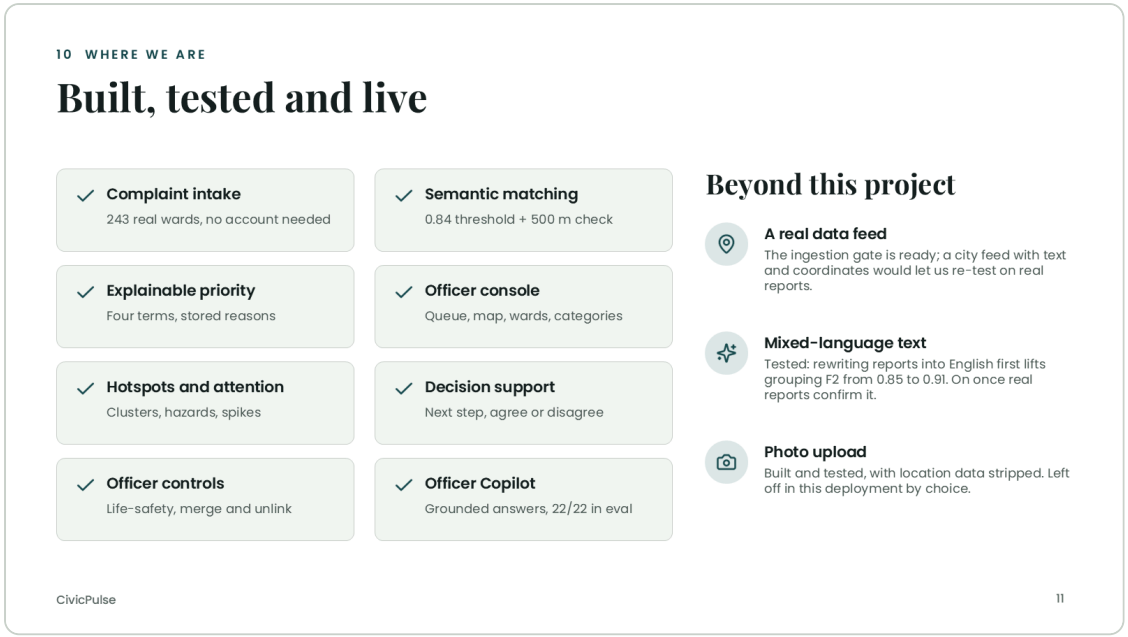
SPEAKER 4 · about 45 seconds

Overall, we measured instead of assuming.

Grouping: 95 percent of real decisions correct. Priority: we had an AI ranker, which never saw our formula, rank 45 incidents three times. Our formula beat the naive “most reports first” baseline every time. Letting age count up to 30 days instead of 14 improved agreement in every run, so we applied it, and that is what is live.

That age-30 calibration is final, and it is what runs on the live site.

SLIDE 11 · WHERE WE ARE



SPEAKER 4 · about 30 seconds

Today, all of this is built, tested and deployed: intake, matching, priority, the console, hotspots, decision support, officer controls and the Copilot.

Beyond this project: the ingestion gate is ready for a real city data feed. We tested mixed Kannada–English text, and rewriting reports into English before matching lifted grouping quality from 0.85 to 0.91; it switches on once real reports confirm it. Photo upload is built, but we left it off in this deployment by choice.

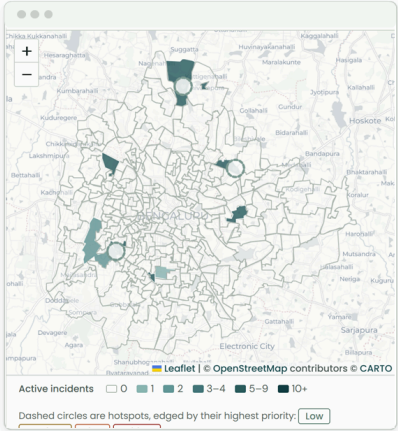
SLIDE 12 · THANK YOU

Thank you

Questions welcome

Try it live
civic-pulse-gamma-ivory.vercel.app

Source
github.com/Proj-CivicPulse/CivicPulse



SPEAKER 4 · about 15 seconds

To sum up: CivicPulse turns scattered complaints into grouped, prioritised incidents, and it can explain every decision it makes.

It is live at the address on screen. Thank you; we are happy to take questions.

(Q&A: each speaker takes questions on their own slides.)

Likely questions

Question	Answer	Who
Why not machine learning for priority?	There is no labelled data to train on, and an officer has to be able to check the answer. A rule with stored reasons can be checked; a model cannot. We tested the rule against an independent ranking instead.	Speaker 3
Why 0.84 and not some other threshold?	It gave the best balance of precision and recall across both datasets: our 144 written test complaints and the real decisions from the live system. At 0.75 everything merged; above about 0.87 we start splitting real duplicates apart.	Speaker 2
What about two different problems at the same spot?	The distance check cannot separate them, but the meaning check does most of the work: a blocked sewer and a missing drain cover read differently. For the rare mistake, the officer can split a report out or merge two incidents, and the system never undoes it.	Speaker 2
Does it handle Kannada and Hindi?	Gemini's embedding model is multilingual, which is why we chose it. We measured one weak spot honestly: two code-mixed complaints about different problems can score as similar as two English ones about the same problem. So we tested rewriting each report into English before matching: grouping F2 went from 0.85 to 0.91. It switches on once real code-mixed reports confirm it.	Speaker 2
Is this real complaint data?	The ward boundaries are real: 243 BBMP wards. The complaints are demo data we generated, because we have no access to a city feed yet. The system has a validated ingestion gate ready for a real feed.	Speaker 4
What does it cost to run?	Each complaint embedding costs a fraction of a rupee. Hosting is Vercel plus Railway.	Speaker 2
Why two backends instead of one?	Spring Boot holds the stable, transactional rules: login, complaints, priority. Node is where the AI work changes quickly. Keeping Spring as the only writer means the fast-moving part can never corrupt the data.	Speaker 2
What happens if Gemini is down?	The citizen never notices. The complaint is saved first; a circuit breaker groups it by ward and category instead, marks it, and a sweep redoes it properly when the service is back.	Speaker 2
Is citizen data sent to an AI?	Complaint text goes to Google's embedding model to compute similarity. The Copilot is designed so that resident complaint text is never sent to the chat model; it answers only from CivicPulse's own figures and records.	Speaker 4
How do you know the priority formula is right?	We tested it against an independent ranking of 45 incidents, three times. It beat "most reports first" every time, and the test told us to let age count up to 30 days, which we applied. That calibration is final.	Speaker 4
Can the Copilot make things up?	Every claim must cite an id from the packet Spring built, and Spring drops any citation that was not in it. In our 22-question test, none of 38 citations and none of 81 numbers came from outside the data.	Speaker 4
Why did the first Copilot test score 21, not 22?	Hotspots were built after the Copilot's data, so asked for hotspots it named the busiest wards: true numbers, wrong definition. We added hotspots and life-safety flags to its data and re-ran the test: 22 of 22.	Speaker 4
Can citizens attach photos?	The feature is built: one photo per report, re-encoded on the server so location and device data are stripped. We left it off in this deployment by choice, since a student project should not store residents' photos.	Speaker 1
Does the life-safety flag change the priority score?	No. Hazard belongs to the problem, not its category, so the flag escalates the incident and puts it in the "needs attention" strip, and the officer confirms or clears it.	Speaker 3